01_std_user_pref.md

Learning & Knowledge Model

- **Pattern recognition over depth**: Learn via high-value correlation triggers, not exhaustive memorization
- **Compression is excellence**: 3.5GB → 5MB; remove noise, preserve function
- **Context navigation > rote knowledge**: Value = ability to move through data intelligently, not amount stored
- **Specific memory anchors**: Tied to concrete clinical decisions/flows, not abstract depth
- **Exam strategy**: Identify highest-yield patterns → drives next action → repeat

Work Style

- **Low cognitive load preference**: Clean, minimal structure; no redundancy; no fluff
- **Structural clarity first**: Naming conventions matter; consistency across projects; clear linking
- **Iterative refinement**: Build → test → compare → fix → iterate (not big-bang delivery)
- **Evidence-based over intuition**: Compare outputs against real data; use gaps to refine rules
- **Swedish clinical + English structural**: Terminology in Swedish; file names, code, architecture in English

Output Preferences

- **JSON for data** (structured, traversable): question banks, cases, entity relations
- **Markdown for narrative/rules**: coverage maps, matrices, audit, documentation
- **No invented structures**: Respect existing naming (`NN_descriptive_name`); extend, don't create parallel systems
- **Playable first, pretty second**: Frontend works before styling; test before polish

What Breaks Work

- Static Q/A without consequence
- Disconnected data (entities not linked back to questions)
- Unstructured sprawl (files without clear purpose or location)
- Depth-focused teaching (contradicts exam strategy)
- Hand-translation between systems (avoid by keeping data clean & format-consistent from start)

Known Strengths

- Pattern matching under time pressure
- Quickly identifying highest-yield clinical flows
- Building clean, minimal data pipelines
- Ruthless compression (keep what matters, delete the rest)

Known Weaknesses

- Exam performance suffers when knowledge is *deep* instead of *patterned*
- Can get lost in unstructured data or sprawling projects
- Need clear, enforced naming to stay organized
- Prefer to see working output early, not theoretical perfection

Cognitive Preferences

- **Think in flows, not trees**: State → decision → consequence → next state
- **Value traversal over storage**: How to move through knowledge matters more than how much is stored
- **Entity linking as core feature**: Questions connect to entities; entities connect to entities; decisions traced via relations
- **Frontend-first validation**: If it doesn't work in the browser, the structure is wrong
- **Minimal viable iteration**: Ship working version, then refine; don't over-engineer before testing